Experiment No. 1

Aim: Implement Linear and Logistic Regression on real-world datasets

1. Dataset Source

Two real-world datasets obtained from **Kaggle** were used for this experiment.

Dataset 1 (Linear Regression)

- **Name:** Global Air Quality Dataset
- **Description:** Contains air pollution and meteorological data collected from urban locations
- **Source:**https://www.kaggle.com/datasets/smeet888/global-air-quality-data15-days-hourly-50-cities
- **File Used:** globalAirQuality.csv

Dataset 2 (Logistic Regression)

- **Name:** Heart Disease Dataset
- **Description:** Contains medical attributes used to predict the presence of heart disease
- **Source Link:** https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset ● **File Used:** heart.csv

2. Dataset Description

Dataset 1: Global Air Quality Dataset

The dataset contains measurements of atmospheric pollutants and environmental parameters.

Features Used:

- pm25 → Particulate Matter 2.5
- pm10 → Particulate Matter 10
- no2 → Nitrogen Dioxide
- so2 → Sulfur Dioxide
- o3 → Ozone
- co → Carbon Monoxide
- temperature
- humidity

- wind_speed

Target Variable (Linear Regression):

- **AQI (Air Quality Index)** – Continuous value

Dataset Size: Approx. 18,000 records

Characteristics:
Real-world environmental data with missing values, which were cleaned using mean imputation.

Dataset 2: Heart Disease Dataset

This dataset contains medical records used for classification.

Features Used:

- age
- sex
- chest pain type
- resting blood pressure
- cholesterol
- maximum heart rate
- ECG results

Target Variable (Logistic Regression):

- **target**
    - 1 → Heart disease present
    - 0 → No heart disease

Dataset Size: Approx. 300 records

Characteristics:
Binary classification dataset with numerical features.

3. Mathematical Formulation of the Algorithms

Linear Regression

Linear Regression predicts a continuous target variable:

y=β0+β1x1+β2x2+⋯+βnxny = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_ny=β0 +β1 x1 +β2x2 + ⋯+βn xn Where:

- yyy → Predicted AQI value
- x1,x2,…,xnx_1, x_2, …, x_nx1 ,x2 ,…,xn → Input features
- β0\beta_0β0 → Intercept
- β1,…,βn\beta_1, …, \beta_nβ1 ,…,βn → Coefficients

Logistic Regression

Logistic Regression predicts a binary outcome using the sigmoid function:

P(y=1)=11+e−zP(y=1) = \frac{1}{1 + e^{-z}}P(y=1)=1+e−z1

Probability is converted to class labels using a threshold of **0.5**:

- 1 → Heart disease present
- 0 → No heart disease

4. Algorithm Limitations

Linear Regression Limitations

- Assumes a linear relationship between features and target
- Sensitive to outliers
- Cannot model complex non-linear relationships

Logistic Regression Limitations

- Assumes a linear decision boundary
- Performance may reduce for imbalanced datasets
- Not suitable for highly non-linear classification problems

5. Methodology / Workflow

Linear Regression Workflow

1. Load globalAirQuality.csv dataset
2. Remove non-numeric columns
3. Handle missing values using mean imputation
4. Select AQI as target variable
5. Split data into training (80%) and testing (20%)
6. Train Linear Regression model
7. Predict AQI values
8. Evaluate using MSE and R^2 score

Logistic Regression Workflow

1. Load heart.csv dataset
2. Separate features and target variable
3. Apply feature scaling
4. Split data into training (80%) and testing (20%)
5. Train Logistic Regression model
6. Predict class labels
7. Evaluate using accuracy and confusion matrix

6. Performance Analysis

Linear Regression Results

- Mean Squared Error (MSE) and R^2 score were calculated.

```
Linear Regression Results
MSE: 221.46771674350427
R2 Score: 0.6265333706498294

Actual vs Predicted AQI (First 10 Values):
   Actual_AQI  Predicted_AQI
0         119      97.926995
1          81     100.360356
2          72      85.782936
3          90     111.188267
4         130     137.567809
5         111     124.838403
6          75      68.634420
7         115     114.450482
8         102     113.782334
9          86      96.214322
```

Interpretation:
The model is able to predict AQI values with moderate accuracy, indicating that pollutant and weather parameters influence air quality.

Logistic Regression Results

- Accuracy, confusion matrix, and classification report were obtained.

```
Logistic Regression Results
Accuracy: 0.7951219512195122

Confusion Matrix:
 [[73 29]
 [13 90]]

Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.72      0.78       102
           1       0.76      0.87      0.81       103

    accuracy                           0.80       205
   macro avg       0.80      0.79      0.79       205
weighted avg       0.80      0.80      0.79       205
```

Interpretation:
The model correctly classifies patients with and without heart disease with good accuracy.

7. Hyperparameter Tuning

- **Linear Regression:**
  Uses a closed-form solution; no hyperparameter tuning required.

- **Logistic Regression:** max_iter = 1000 was used to ensure convergence. Regularization parameter **C** can be tuned if required.

CODE-

```
import pandas as pd import matplotlib.pyplot as plt from

sklearn.model_selection import train_test_split from

sklearn.linear_model import LinearRegression from

sklearn.metrics import mean_squared_error, r2_score

import warnings

warnings.filterwarnings("ignore")

# Load dataset df =

pd.read_csv("globalAirQuality.csv")

# Drop non-numeric columns df.drop(['timestamp',

'country', 'city'], axis=1, inplace=True)

# Convert to numeric & handle missing values

df = df.apply(pd.to_numeric, errors='coerce')

df.fillna(df.mean(), inplace=True)


# Rename target column
df.rename(columns={'aqi': 'AQI'}, inplace=True)


# Features & target X =

df.drop('AQI', axis=1) y =

df['AQI']
```

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model model =
LinearRegression()
model.fit(X_train, y_train)
# Prediction y_pred =
model.predict(X_test)

# Evaluation print("Linear Regression Results")
print("MSE:", mean_squared_error(y_test,
y_pred)) print("R2 Score:", r2_score(y_test,
y_pred))

# Print values in terminal
comparison = pd.DataFrame({
    "Actual_AQI": y_test.values[:10],     "Predicted_AQI": y_pred[:10]
})
print("\nActual vs Predicted AQI (First 10 Values):") print(comparison)
```

Logistic_regression- import pandas as pd import matplotlib.pyplot as plt from sklearn.model_selection import train_test_split from sklearn.linear_model import LogisticRegression from sklearn.preprocessing import StandardScaler from

```
sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import warnings warnings.filterwarnings("ignore")

# Load dataset df =

pd.read_csv("heart.csv")

 # Features & target X = df.drop('target', axis=1) y = df['target']

 # Feature scaling scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(     X_scaled, y, test_size=0.2,
random_state=42  )

# Train model model =

LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

# Prediction y_pred =

model.predict(X_test)

# Evaluation print("Logistic Regression Results")

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:\n", confusion_matrix(y_test,

y_pred)) print("\nClassification Report:\n",

classification_report(y_test, y_pred))
```

Conclusion

In this experiment, Linear Regression was implemented on a real-world air quality dataset to predict AQI values, and Logistic Regression was implemented on a heart disease dataset to perform binary classification. The experiment demonstrates practical application of regression techniques, data preprocessing, model training, and performance evaluation using Python.